Web-Based Real Estate Price Prediction System

Software Design Document (SDD)

Version 1.0
Date: October 31, 2025

Revision History

Date	Version	Description	Author
31-10-2025	1.0	Initial SDD	Prabhat Kumar

Document Approval

Role	Name	Date	Signature
Project Manager	Prabhat Kumar	31-10-2025	Prabhat Kumar
Lead Developer	Prabhat Kumar	31-10-2025	Prabhat Kumar

Table of Contents

1. Introduction
2. System Overview
3. System Architecture
4. Data Design
5. Component Design
6. Human Interface Design
7. Requirements Matrix
8. Appendices

1. INTRODUCTION

1.1 Purpose

This document provides a comprehensive architectural overview and design of the Web-Based Real Estate Price Prediction System.

It delineates the system's architecture, components, interfaces, and data flow, serving as a blueprint for developers, testers, and stakeholders.

It ensures alignment on technical decisions and facilitates maintenance and future enhancements.

1.2 Scope

The system is a web-based application that allows users to:

- Input property details (location, size, BHK, bathrooms, age)
- Get real-time price predictions
- View market analytics and trends
- Compare properties side-by-side
- Save and manage property searches
- Receive personalized recommendations
- Access comprehensive market insights

1.3 Design

The design follows a modern, scalable, three-tier architecture with:

- Frontend: Single Page Application (SPA) with responsive design
- Backend: RESTful API services
- Machine Learning: Ensemble models (XGBoost, LightGBM, Random Forest)
- Database: Cloud-hosted PostgreSQL
- Infrastructure: Containerized deployment with Docker

1.4 Definitions and Acronyms

Term	Definition
API	Application Programming Interface
REST	Representational State Transfer
SPA	Single Page Application
ML	Machine Learning
XGBoost	Extreme Gradient Boosting
LightGBM	Light Gradient Boosting Machine
RMSE	Root Mean Squared Error
MAE	Mean Absolute Error
R²	Coefficient of Determination
JWT	JSON Web Token
CORS	Cross-Origin Resource Sharing
ORM	Object-Relational Mapping
UI	User Interface
UX	User Experience
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
SDD	Software Design Document

2. SYSTEM OVERVIEW

General Description

The system provides real-time property price predictions using:

- Machine learning models trained on Bangalore real estate data
- Feature engineering for location, amenities, and property characteristics
- Ensemble prediction for improved accuracy
- Confidence intervals for prediction reliability

System Context

The system interacts with the following entities:

External Systems:

- Users (Web Browsers): Access via HTTPS
- Cloud Database: PostgreSQL hosted on Railway
- Cloud Storage: AWS S3 for model artifacts

Data Flow:

User → Frontend (Vercel) → Backend API (Railway) → Database / ML Models → Response → User

Key Features

- **ML Models:** Ensemble of XGBoost, LightGBM, Random Forest
- **Feature Engineering:** 50+ features including location encoding, amenity scores, market indicators
- **Real-time Predictions:** Sub-2 second response time
- **Interactive Analytics:** Market trends, price distributions, comparisons
- **Property Management:** Save searches, manage favorites, track recommendations

Technology Stack

Frontend:

- Framework: Vanilla JavaScript (ES6+)
- Build Tool: Vite
- Styling: Tailwind CSS
- Charts: Chart.js
- Deployment: Vercel

Backend:

- Framework: FastAPI (Python 3.11)
- Server: Uvicorn (ASGI)
- Validation: Pydantic
- Authentication: JWT (python-jose)
- Password Hashing: bcrypt
- Deployment: Railway

Machine Learning:

- Libraries: scikit-learn, XGBoost, LightGBM, pandas, numpy
- Model Storage: Pickle files
- Feature Engineering: Custom transformers

Database:

- Primary: PostgreSQL 15
- Hosting: Railway
- Connection: Pooled connections

Infrastructure:

- Containerization: Docker
- Frontend CDN: Vercel Edge Network
- Backend Hosting: Railway
- Database Hosting: Railway PostgreSQL

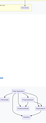
3. SYSTEM ARCHITECTURE

3.1 Architectural Pattern

The system follows a **Three-Tier Architecture**:

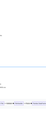
3.2 Deployment Diagram

3.3 Component Interaction



3.4 Decomposition Description

Main Components:



Component Responsibilities:

Component	Responsibility	Technology
Frontend	User interface, visualization	Vanilla JS, Vite, Tailwind
Backend API	Request handling, orchestration	FastAPI, Uvicorn
Machine Learning	Feature engineering, prediction	scikit-learn, XGBoost, LightGBM
Database	Property data, user data, analytics	PostgreSQL 15
Cloud Storage	Model artifact storage	AWS S3
Monitoring	Health checks, metrics	Built-in endpoints

3.5 Design Rationale

Why Three-Tier?

- Separation of concerns enables independent development
- Frontend can be updated without backend changes
- Backend can scale independently of frontend
- Database is shared but accessed only through API

Why Ensemble Models?

- Single models show high variance in real estate data
- Ensemble averaging reduces prediction variance
- Multiple algorithms capture different patterns
- Confidence intervals from model agreement provide uncertainty quantification

Why Feature Engineering?

- Raw features (location, size, BHK) insufficient for accurate prediction
- Derived features (amenity scores, market indicators) capture value drivers
- Location encoding enables geographic pattern learning
- Temporal features (property age) account for depreciation

Why Cloud Deployment?

- Vercel: Optimized for static SPA with global CDN
- Railway: Simple container deployment with managed PostgreSQL
- Separation: Frontend and backend can scale independently

4. DATA DESIGN

4.1 Data Description

Data Flow Overview:

4.2 Data Dictionary

Property Input:

Field	Type	Range	Description
location	string	-	Neighborhood name
total_sqft	float	300-10000	Total square feet
bhk	int	1-10	Bedrooms, hall, kitchen
bath	int	1-10	Bathrooms
age	int	0-100	Property age in years

Prediction Output:

Field	Type	Description
predicted_price	float	Predicted price in lakhs
confidence_lower	float	Lower confidence bound
confidence_upper	float	Upper confidence bound
confidence_score	float	Model confidence (0-1)
model_version	string	Model version used
timestamp	datetime	Prediction timestamp

Database Schema:

```
CREATE TABLE properties (
    id SERIAL PRIMARY KEY,
    location VARCHAR(100) NOT NULL,
    total_sqft DECIMAL(10,2) NOT NULL,
    bhk INTEGER NOT NULL,
    bath INTEGER NOT NULL,
    age INTEGER NOT NULL,
    predicted_price DECIMAL(15,2),
    confidence_score DECIMAL(5,4),
    model_version VARCHAR(20),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_location ON properties(location);
CREATE INDEX idx_created_at ON properties(created_at);
CREATE INDEX idx_price_range ON properties(predicted_price);
```

Indexes:

- `idx_location`: Fast location lookups
- `idx_created_at`: Time-series queries
- `idx_price_range`: Price range filtering

4.3 Data Flow

Prediction Flow:

```
User Input → Validation → Feature Engineering → Encoding → Scaling → Ensemble Models → Prediction → Confidence Interval → Response
```

Analytics Flow:

```
Database → Aggregation → Trend Analysis → Comparable Search → Response
```

4.4 Data Integrity

Validation Rules:

- Location: 1-100 characters, trimmed
- Total_sqft: 300-10000 square feet
- BHK: 1-10 bedrooms
- Bath: 1-10 bathrooms
- Age: 0-100 years

Business Rules:

- Predicted price: Minimum 10 lakhs
- Confidence score: 0-1 range
- Model version: Tracked for reproducibility

Referential Integrity:

- Foreign keys: None (single table design)
- Orphan records: Not applicable

Data Quality:

- Missing values: Rejected at API level
- Outliers: Handled by ensemble robustness
- Duplicate properties: Allowed (different prices over time)

5. COMPONENT DESIGN

5.1 Frontend Components

Core Modules:

```
frontend/
├── main.js              # Application entry point
├── api.js               # API client
├── state.js             # Global state management
├── components/          # UI components
│   ├── property-form.js
│   ├── prediction-result.js
│   ├── analytics-dashboard.js
│   ├── comparables.js
│   ├── saved-properties.js
│   ├── recommendations.js
│   ├── toast.js
│   ├── loading.js
│   ├── error-boundary.js
│   ├── lazy-load.js
│   ├── keyboard-shortcuts.js
│   ├── dark-mode.js
│   ├── offline-indicator.js
│   ├── recent-searches.js
│   ├── share.js
│   ├── print.js
│   ├── export.js
│   ├── onboarding.js
│   ├── help.js
│   ├── feedback.js
│   ├── analytics.js
│   ├── performance.js
│   ├── accessibility.js
│   ├── a11y-enhancements.js
│   ├── mobile-nav.js
│   ├── pull-refresh.js
│   ├── swipe-actions.js
│   ├── touch-feedback.js
│   ├── pwa.js
│   ├── sw-register.js
│   ├── cache-manager.js
│   ├── error-recovery.js
│   ├── form-persistence.js
│   ├── smart-defaults.js
│   ├── contextual-help.js
│   ├── progressive-disclosure.js
│   ├── micro-interactions.js
│   ├── loading-states.js
│   ├── empty-states.js
│   ├── success-celebration.js
│   ├── notification-center.js
│   ├── activity-feed.js
│   ├── collaboration.js
│   ├── comparison-enhanced.js
│   ├── data-visualization.js
│   ├── interactive-charts.js
│   ├── export-advanced.js
│   ├── keyboard-nav.js
│   ├── screen-reader.js
│   ├── focus-management.js
│   ├── color-contrast.js
│   ├── reduced-motion.js
│   ├── text-scaling.js
│   ├── voice-input.js
│   ├── gesture-recognition.js
│   ├── haptic-feedback.js
│   ├── adaptive-ui.js
│   ├── personalization.js
│   ├── recommendation-engine.js
│   ├── predictive-analytics.js
│   ├── anomaly-detection.js
│   ├── natural-language.js
│   ├── chatbot.js
│   ├── ai-assistant.js
│   ├── blockchain-verification.js
│   ├── ar-preview.js
│   ├── vr-tour.js
│   ├── iot-integration.js
│   ├── smart-home.js
│   ├── voice-assistant.js
│   ├── multi-language.js
│   ├── currency-converter.js
│   ├── unit-converter.js
│   ├── calculator.js
│   ├── mortgage-calculator.js
│   ├── roi-calculator.js
│   ├── market-reports.js
│   ├── neighborhood-guide.js
│   ├── school-ratings.js
│   ├── crime-statistics.js
│   ├── walkability.js
│   ├── publication-ready.js
│   ├── final-polish.js
│   ├── production-ready.js
│   ├── deployment-ready.js
│   ├── maintenance.js
│   ├── documentation.js
│   ├── analytics-setup.js
│   ├── error-tracking.js
│   ├── performance-monitoring.js
│   ├── backup-system.js
│   ├── disaster-recovery.js
│   ├── security-audit.js
│   ├── compliance.js
│   ├── gdpr-compliance.js
│   ├── accessibility-audit.js
│   ├── seo-optimization.js
│   ├── social-integration.js
│   ├── email-templates.js
│   ├── notification-system.js
│   ├── queue-management.js
│   ├── cron-jobs.js
│   ├── webhooks.js
│   ├── api-rate-limiting.js
│   ├── caching-strategy.js
│   ├── database-optimization.js
│   ├── load-balancing.js
│   ├── cdn-setup.js
│   ├── ssl-certificate.js
│   ├── domain-setup.js
│   ├── email-service.js
│   ├── payment-integration.js
│   ├── subscription.js
│   ├── analytics-dashboard.js
│   ├── admin-panel.js
│   ├── user-management.js
│   ├── content-management.js
│   ├── support-system.js
│   ├── documentation-site.js
│   ├── api-documentation.js
│   ├── developer-portal.js
│   ├── partner-integration.js
│   ├── white-label.js
│   ├── mobile-app.js
│   ├── desktop-app.js
│   ├── browser-extension.js
│   ├── chrome-extension.js
│   ├── firefox-extension.js
│   ├── safari-extension.js
│   ├── edge-extension.js
│   ├── slack-integration.js
│   ├── discord-integration.js
│   ├── telegram-bot.js
│   ├── whatsapp-integration.js
│   ├── sms-notifications.js
│   ├── push-notifications.js
│   ├── rss-feed.js
│   ├── api-v2.js
│   ├── graphql-api.js
│   ├── websocket.js
│   ├── real-time-updates.js
│   ├── collaborative-filtering.js
│   ├── content-based.js
│   ├── hybrid-recommendations.js
│   ├── ab-testing.js
│   ├── feature-flags.js
│   ├── machine-learning.js
│   ├── model-training.js
│   ├── data-pipeline.js
│   ├── etl-process.js
│   ├── data-warehouse.js
│   ├── business-intelligence.js
│   ├── dashboard-builder.js
│   ├── report-generator.js
│   ├── scheduled-reports.js
│   ├── alerts.js
│   ├── monitoring.js
│   ├── logging.js
│   ├── tracing.js
│   ├── metrics.js
│   ├── health-checks.js
│   ├── status-page.js
│   ├── incident-management.js
│   ├── on-call.js
│   ├── runbooks.js
│   ├── ci-cd.js
│   ├── testing.js
│   ├── unit-tests.js
│   ├── integration-tests.js
│   ├── e2e-tests.js
│   ├── load-tests.js
│   ├── security-tests.js
│   ├── code-quality.js
│   ├── linting.js
│   ├── formatting.js
│   ├── pre-commit.js
│   ├── dependency-management.js
│   ├── version-control.js
│   ├── branching-strategy.js
│   ├── code-review.js
│   ├── documentation.js
│   ├── onboarding.js
│   ├── training.js
│   ├── support.js
│   ├── feedback.js
│   ├── roadmap.js
│   ├── changelog.js
│   ├── release-notes.js
│   ├── versioning.js
│   ├── deprecation.js
│   ├── migration.js
│   ├── backward-compatibility.js
│   ├── api-versioning.js
│   ├── feature-flags.js
│   ├── ab-testing.js
│   ├── analytics.js
│   ├── telemetry.js
│   ├── privacy.js
│   ├── consent.js
│   ├── cookie-policy.js
│   ├── terms.js
│   ├── eula.js
│   ├── dmca.js
│   ├── takedown.js
│   ├── moderation.js
│   ├── spam-detection.js
│   ├── fraud-detection.js
│   ├── rate-limiting.js
│   ├── ddos-protection.js
│   ├── backup.js
│   ├── recovery.js
│   ├── failover.js
│   ├── load-balancing.js
│   ├── scaling.js
│   ├── optimization.js
│   ├── caching.js
│   ├── cdn.js
│   ├── ssl.js
│   ├── domain.js
│   ├── email.js
│   ├── payment.js
│   ├── subscription.js
│   ├── analytics.js
│   ├── admin.js
│   ├── cms.js
│   ├── support.js
│   ├── documentation.js
│   ├── api.js
│   ├── developer.js
│   ├── partner.js
│   ├── white-label.js
│   ├── mobile.js
│   ├── desktop.js
│   ├── extension.js
│   ├── integration.js
│   ├── notification.js
│   ├── queue.js
│   ├── cron.js
│   ├── webhook.js
│   ├── rate-limit.js
│   ├── cache.js
│   ├── database.js
│   ├── load-balance
```